```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import mean_squared_error, accuracy_score
from sklearn.utils.class_weight import compute_class_weight
```

```
train = pd.read_csv("train_final.csv")
test = pd.read_csv("test_final.csv")

print(train.shape)
print(test.shape)
```

```
(20631, 21)
(13096, 21)
```

```
train["RUL"] = train["RUL"].fillna(train["RUL"].median())
test["RUL"] = test["RUL"].fillna(test["RUL"].median())

train["RUL"] = train["RUL"].clip(upper=125)
test["RUL"] = test["RUL"].clip(upper=125)
```

```
def create_fault_stage(rul):

    if rul > 40:
        return 0   # Healthy

    elif rul > 15:
        return 1   # Warning

    else:
        return 2   # Critical


train["fault_stage"] = train["RUL"].apply(create_fault_stage)
test["fault_stage"] = test["RUL"].apply(create_fault_stage)

print(train["fault_stage"].value_counts())
```

```
fault_stage
0    16531
1     2500
2     1600
Name: count, dtype: int64
```

```
FEATURES = [
    c for c in train.columns
    if c not in ["unit_id", "cycle", "RUL", "fault_stage"]
]

train[FEATURES] = train[FEATURES].fillna(0)
test[FEATURES] = test[FEATURES].fillna(0)

print(FEATURES)
print(len(FEATURES))
```

```
['op_setting_1', 'op_setting_2', 'op_setting_3', 's2', 's3', 's4', 's6', 's7', 's8', 's9', 's11', 's12', 's13', 's14', 's15', 's
18
```

```
WINDOW_SIZE = 30


def create_sequences(df, features):

    X, y_rul, y_stage = [], [], []

    for unit in df["unit_id"].unique():

        unit_df = df[df["unit_id"] == unit].sort_values("cycle")
```

```
        feature_values = unit_df[features].values
        rul_values = unit_df["RUL"].values
        stage_values = unit_df["fault_stage"].values

        for i in range(len(unit_df) - WINDOW_SIZE + 1):

            X.append(feature_values[i:i+WINDOW_SIZE])
            y_rul.append(rul_values[i+WINDOW_SIZE-1])
            y_stage.append(stage_values[i+WINDOW_SIZE-1])

    return np.array(X), np.array(y_rul), np.array(y_stage)
```

```
X_train, y_rul_train, y_stage_train = create_sequences(train, FEATURES)
X_test, y_rul_test, y_stage_test = create_sequences(test, FEATURES)

y_rul_train = y_rul_train / 125.0
y_rul_test = y_rul_test / 125.0
```

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)

y_rul_train_t = torch.tensor(y_rul_train, dtype=torch.float32)
y_rul_test_t = torch.tensor(y_rul_test, dtype=torch.float32)

y_stage_train_t = torch.tensor(y_stage_train, dtype=torch.long)
y_stage_test_t = torch.tensor(y_stage_test, dtype=torch.long)


train_loader = DataLoader(
    TensorDataset(X_train_t, y_rul_train_t, y_stage_train_t),
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_rul_test_t, y_stage_test_t),
    batch_size=64,
    shuffle=False
)
```

```
classes = np.unique(y_stage_train)

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_stage_train
)

class_weights = torch.tensor(weights, dtype=torch.float32)

print(class_weights)
```

```
tensor([0.4336, 2.3641, 3.6940])
```

```
class Attention(nn.Module):

    def __init__(self, hidden_dim):
        super().__init__()
        self.attn = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        scores = self.attn(x)
        weights = torch.softmax(scores, dim=1)
        context = torch.sum(weights * x, dim=1)
        return context


class AttentionLSTM(nn.Module):

    def __init__(self, input_dim):
        super().__init__()

        self.lstm = nn.LSTM(
```

```
            input_size=input_dim,
            hidden_size=128,
            num_layers=2,
            batch_first=True,
            dropout=0.2
        )

        self.attention = Attention(128)

        self.rul_head = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1)
        )

        self.stage_head = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 3)
        )

    def forward(self, x):

        lstm_out, _ = self.lstm(x)

        context = self.attention(lstm_out)

        rul = self.rul_head(context).squeeze(1)
        stage = self.stage_head(context)

        return rul, stage
```

```
device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu"
)

model = AttentionLSTM(
    input_dim=len(FEATURES)
).to(device)

optimizer = optim.Adam(
    model.parameters(),
    lr=0.0003
)

rul_loss_fn = nn.HuberLoss(delta=1.0)

stage_loss_fn = nn.CrossEntropyLoss(
    weight=class_weights.to(device)
)
```

```
def train_one_epoch():

    model.train()

    total_loss_epoch = 0

    for Xb, yr, ys in train_loader:

        Xb = Xb.to(device)
        yr = yr.to(device)
        ys = ys.to(device)

        optimizer.zero_grad()

        rul_pred, stage_pred = model(Xb)

        rul_loss = rul_loss_fn(rul_pred, yr)
        stage_loss = stage_loss_fn(stage_pred, ys)

        total_loss = rul_loss + 0.5 * stage_loss

        total_loss.backward()
```

```
            torch.nn.utils.clip_grad_norm_(
                model.parameters(),
                max_norm=0.5
            )

            optimizer.step()

            total_loss_epoch += total_loss.item()

        return total_loss_epoch / len(train_loader)
```

```
    def evaluate():

        model.eval()

        rul_preds = []
        rul_true = []

        stage_preds = []
        stage_true = []

        with torch.no_grad():

            for Xb, yr, ys in test_loader:

                Xb = Xb.to(device)

                rp, sp = model(Xb)

                rul_preds.extend(rp.cpu().numpy())
                rul_true.extend(yr.numpy())

                stage_preds.extend(
                    torch.argmax(sp, dim=1).cpu().numpy()
                )

                stage_true.extend(ys.numpy())

        rul_preds = np.array(rul_preds) * 125
        rul_true = np.array(rul_true) * 125

        rmse = np.sqrt(
            mean_squared_error(
                rul_true,
                rul_preds
            )
        )

        acc = accuracy_score(
            stage_true,
            stage_preds
        )

        return rmse, acc
```

```
    EPOCHS = 20

    best_rmse = float("inf")
    best_acc = 0

    patience = 5
    counter = 0

    for epoch in range(EPOCHS):

        train_loss = train_one_epoch()

        rmse, acc = evaluate()

        if rmse < best_rmse:

            best_rmse = rmse
            best_acc = acc

            counter = 0
```

```
            torch.save(
                model.state_dict(),
                "best_attention_lstm_3class.pth"
            )

            print("Best model saved.")

        else:
            counter += 1

        print(
            f"Epoch {epoch+1}/{EPOCHS} | "
            f"Loss: {train_loss:.4f} | "
            f"RMSE: {rmse:.4f} | "
            f"Acc: {acc:.4f}"
        )

        if counter >= patience:
            print("Early stopping triggered.")
            break


    print("\nBest RMSE:", best_rmse)
    print("Best Accuracy:", best_acc)
```

```
Best model saved.
Epoch 1/20 | Loss: 0.2854 | RMSE: 45.8170 | Acc: 0.7274
Best model saved.
Epoch 2/20 | Loss: 0.1534 | RMSE: 45.3497 | Acc: 0.7643
Epoch 3/20 | Loss: 0.1397 | RMSE: 49.9225 | Acc: 0.7487
Epoch 4/20 | Loss: 0.1241 | RMSE: 48.2927 | Acc: 0.7735
Epoch 5/20 | Loss: 0.1158 | RMSE: 46.7248 | Acc: 0.7692
Best model saved.
Epoch 6/20 | Loss: 0.1042 | RMSE: 45.0354 | Acc: 0.8402
Best model saved.
Epoch 7/20 | Loss: 0.1055 | RMSE: 44.6945 | Acc: 0.7597
Epoch 8/20 | Loss: 0.0947 | RMSE: 45.4142 | Acc: 0.7641
Epoch 9/20 | Loss: 0.1039 | RMSE: 45.1350 | Acc: 0.7901
Best model saved.
Epoch 10/20 | Loss: 0.0950 | RMSE: 44.3187 | Acc: 0.7733
Best model saved.
Epoch 11/20 | Loss: 0.0902 | RMSE: 43.8614 | Acc: 0.8096
Epoch 12/20 | Loss: 0.0890 | RMSE: 46.3135 | Acc: 0.7948
Epoch 13/20 | Loss: 0.0928 | RMSE: 44.6009 | Acc: 0.8295
Best model saved.
Epoch 14/20 | Loss: 0.0801 | RMSE: 43.3470 | Acc: 0.8105
Epoch 15/20 | Loss: 0.0809 | RMSE: 45.5678 | Acc: 0.8168
Epoch 16/20 | Loss: 0.0790 | RMSE: 44.9236 | Acc: 0.8129
Epoch 17/20 | Loss: 0.0736 | RMSE: 45.7088 | Acc: 0.7896
Epoch 18/20 | Loss: 0.0645 | RMSE: 47.3535 | Acc: 0.7884
Epoch 19/20 | Loss: 0.0667 | RMSE: 43.6661 | Acc: 0.8437
Early stopping triggered.

Best RMSE: 43.34695908947579
Best Accuracy: 0.810513927030208
```

```
model = AttentionLSTM(
    input_dim=len(FEATURES)
).to(device)

model.load_state_dict(
    torch.load(
        "best_attention_lstm_3class.pth"
    )
)

model.eval()
```

```
AttentionLSTM(
  (lstm): LSTM(18, 128, num_layers=2, batch_first=True, dropout=0.2)
  (attention): Attention(
    (attn): Linear(in_features=128, out_features=1, bias=True)
  )
  (rul_head): Sequential(
    (0): Linear(in_features=128, out_features=64, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.2, inplace=False)
    (3): Linear(in_features=64, out_features=1, bias=True)
  )
```

```
      (stage_head): Sequential(
        (0): Linear(in_features=128, out_features=64, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.2, inplace=False)
        (3): Linear(in_features=64, out_features=3, bias=True)
      )
    )
```